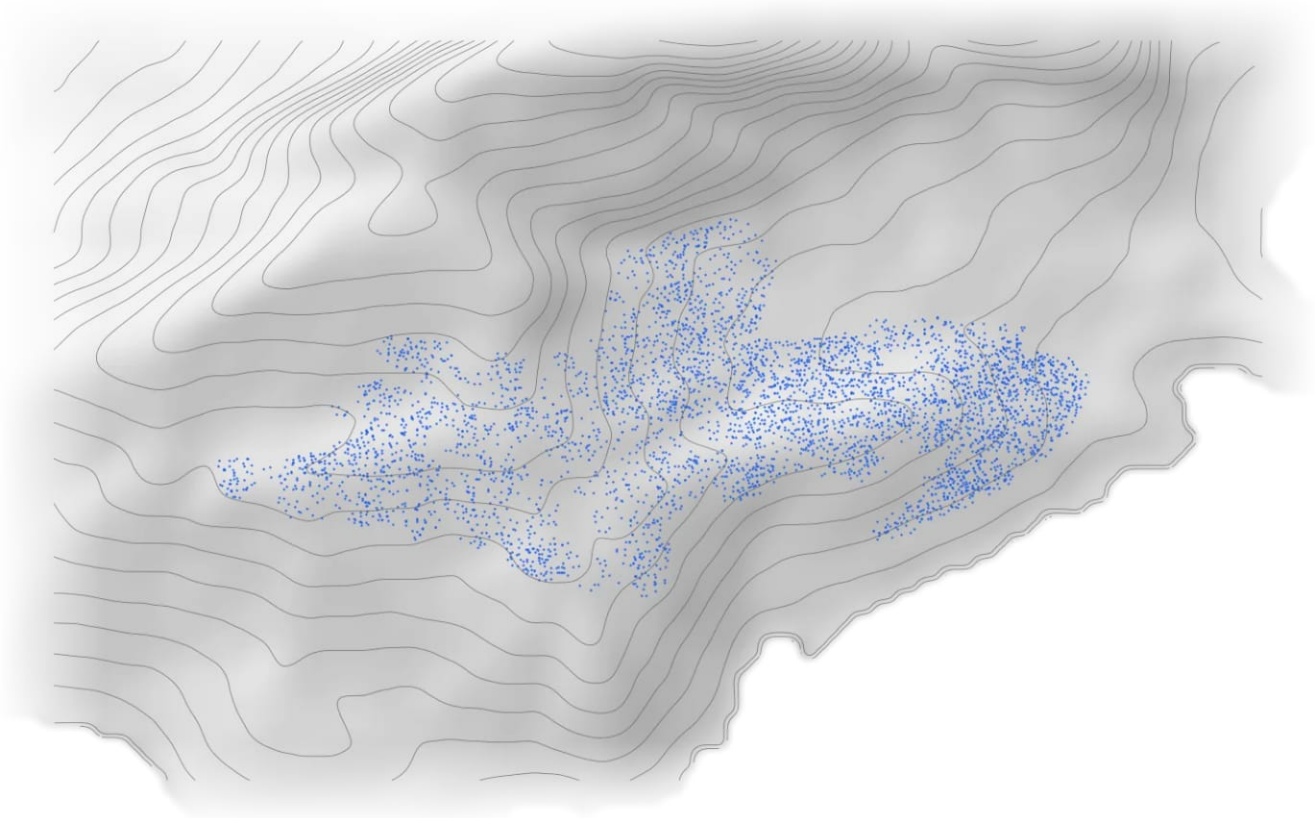
PORTFOLIO · OCTOBER 2026

I make hidden structure visible.

I study cities like a roboticist would. Scan. Model. Decipher.



Théo Alessandro Hermann

Roboticist and urban scientist · MIT Senseable City Lab Rio

thermann.ai@gmail.com · theoalessandro.com · theohermann.ch · github.com/thrmnn · linkedin.com/in/theohermann-epfl

ABOUT

I'm Théo, a French-Brazilian roboticist and urban scientist. Paris, São Paulo, Rio, Lausanne, Amsterdam, Barcelona, Stockholm.

I trained in robotics at EPFL, built perception for a person-following robot at VITA Lab and for an autonomous boat at Roboat, then brought the same tools to cities at MIT Senseable City Lab.

Wherever I work, the question is the same: what aren't we seeing yet that explains how the system behaves? Today I help robot makers find out why their systems fail, and I study how city form shapes light and air.

Speaking Dubai Future Week, 18–21 Nov 2026

PRACTICE · 2026

Making autonomous systems legible

Hidden	Robot failures the logs don't explain.
Revealed	The real cause, found and proved independently.
Changed	Evidence a team can act on, and that its safety assessors will ask for from 2027.

theohermann.ch

IN THIS PORTFOLIO

Camera perception for canal boats Roboat, MIT spinoff · first intern, 2024 3

How city form shapes light and air MIT Senseable City Lab Rio · since 2025 4

Leaf area from an aerial scan Senseable City Lab, MIT and AMS Institute · since 2024 5

Finding the moment a robot got lost Open source · 2026 6

Autonomous mobile robot navigation on a Segway Loomo EPFL VITA Lab · 2023 8

A real-time vision pipeline Open source · 2026 8

Event cameras for high-speed drone state estimation IRI (CSIC-UPC, Barcelona) · 2023–24 9

A harness for AI agents Open source · 2026 9

Roboat, MIT spinoff · first intern, 2024

Camera perception for canal boats

Hidden	An autonomous boat that read the canals through LiDAR alone.
Revealed	Camera detection and tracking, running on the boat's own edge computer.
Changed	A second sense on board, and every model traceable to the recordings it learned from.

THE PROBLEM

Roboat's autonomy already worked, and it worked on LiDAR. LiDAR is precise and expensive, and on water it reports a great deal that is not an obstacle: reflection, wake, chop. The open question when I arrived was how much of the perception job a camera could take on instead, on hardware that fits on a small vessel and inside a real-time budget.

WHAT I DID

I joined as the first intern and built the camera side of the stack: detection and multi-object tracking running on the vessel's own edge hardware, trained on the recordings the boats had been collecting from the canals. Water is an awkward subject for a detector, because the same object looks different under glare, reflection and weather, and the things you most need to catch are the things you have least footage of.

The second half of the work was the part that outlives any particular model. Datasets and model versions went under DVC so a result could be traced back to the exact data that produced it, and retraining stopped being a thing one person could do on one laptop.

WHAT IS NOT HERE

The performance figures belong to Roboat and are not mine to publish.

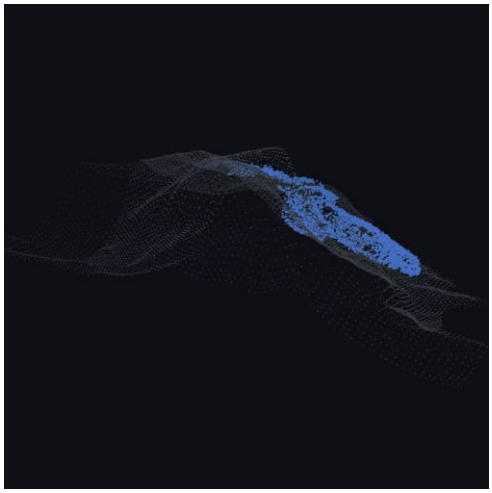
Object Detection · Sensor Fusion · Multi-Object Tracking · PyTorch · YOLO · ROS 2 · Jetson Orin · Edge Deployment

theoalessandro.com/projects/autonomous-boat-perception

MIT Senseable City Lab Rio · since 2025

How city form shapes light and air

Hidden	How light and air move through a city built without plans, which measures made for street grids miss.
Revealed	3D models of the fabric, from the whole city down to a single alley, to study sunlight and airflow where people live.
Changed	Measures that hold on a fabric the standard ones were not built for. The findings stay with the papers until they are out.



THE PROBLEM

Most ways of describing a city assume that someone drew it first: streets with names, plots with edges, blocks that repeat. Informal settlements are not built that way, so the standard descriptors either fail outright or quietly measure the wrong thing. Anything you want to say afterwards about how air, light or heat move through that fabric depends on describing its geometry first.

WHAT I BUILD

A Python pipeline that turns building footprints and a digital terrain model into a grid of morphometric indicators, fine enough that a single alley is resolved rather than averaged away. The indicators are the standard vocabulary of urban physics, computed so they hold on a fabric they were not designed for: sky openness, frontal area density, height variation, slope, aspect, porosity.

On top of that sits a sampling design that decides where a computational fluid dynamics campaign should actually look, rather than simulating everywhere at uniform cost, and an ingestion layer that turns public meteorological records into the boundary conditions those simulations need. The simulations themselves run elsewhere; this work defines what they receive and how their output is read back.

WHAT IS SHOWN HERE

The geometry of one site, Vidigal, is drawn on this site's front page. Nothing else from this work appears here. It is unpublished research with co-authors, so this page describes the method and stops there: no indicators, no findings, no figures from the study.

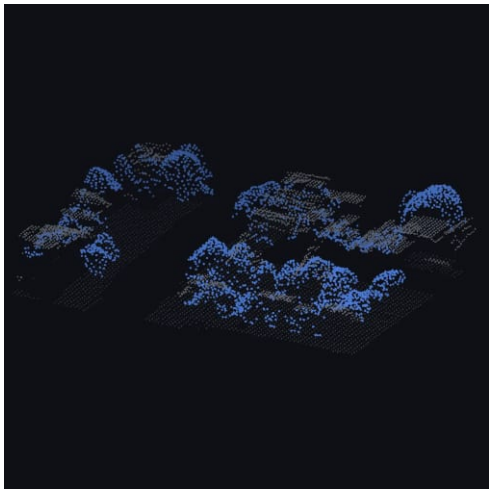
Urban morphometrics · Point clouds · LiDAR · Terrain models · Python

theoalessandro.com/projects/urban-morphometrics

Senseable City Lab, MIT and AMS Institute · since 2024

Leaf area from an aerial scan

Hidden	How much leaf a city's trees carry. Registers record where each tree stands, not how much shade it gives.
Revealed	Leaf area per tree, recovered by a physical inversion from the aerial LiDAR survey the city already flies.
Changed	An estimate that can be argued with on its own terms, from data that already exists. The numbers stay with the paper until it is out.



THE PROBLEM

Municipal tree registers are good at the things a register is for: species, location, trunk diameter, when it was last inspected. They do not record how much leaf a tree actually carries, which is the quantity that matters for shade, cooling and interception. Measuring it directly, tree by tree, across a city is not something anyone is going to fund.

WHAT I BUILD

Cities already fly aerial LiDAR surveys on a regular cycle for entirely unrelated reasons. Those scans contain the information: a pulse that passes through a canopy and a pulse that is intercepted by it are distinguishable, so the fraction of sky visible through the crown can be recovered from data that already exists.

From that gap fraction, leaf area follows by a physical inversion rather than a fitted model, which is the part I care about most. The relationship between how much light a canopy blocks and how much leaf it carries is classical physics with known assumptions, so the estimate can be argued about on its own terms instead of defended as the output of a regressor nobody can inspect. An earlier version of this work did use machine learning; the current method does not.

WHAT IS SHOWN HERE

Nothing but the method. This is a multi-author manuscript in preparation, so there are no numbers, figures or findings on this page, and there will not be until it is published.

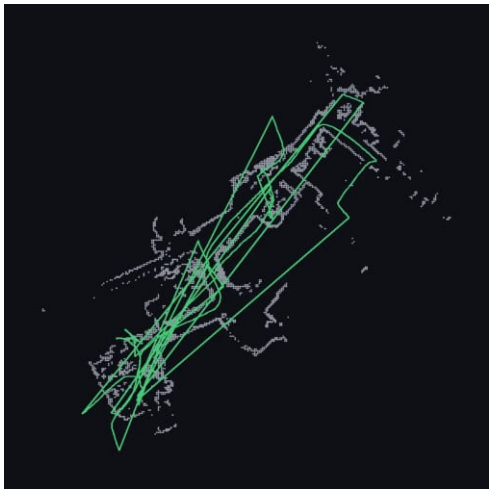
Remote sensing · LiDAR · Canopy · Beer-Lambert inversion · Python

theoalessandro.com/projects/aerial-lidar-tree-census

Open source · 2026

Finding the moment a robot got lost

Hidden	Robots that lose track of where they are while still reporting high confidence.
Revealed	Four detectors that find those moments in the logs a fleet already records, run unchanged on recordings from five platforms.
Changed	Published with what it missed, and nearly every number recomputes from the repository with one script.



Method public: github.com/thrmnn/ros2-localization-triage

PROBLEM

A robot misbehaves at a customer site and someone has to comb the bag files and logs to find out why.

BUILT

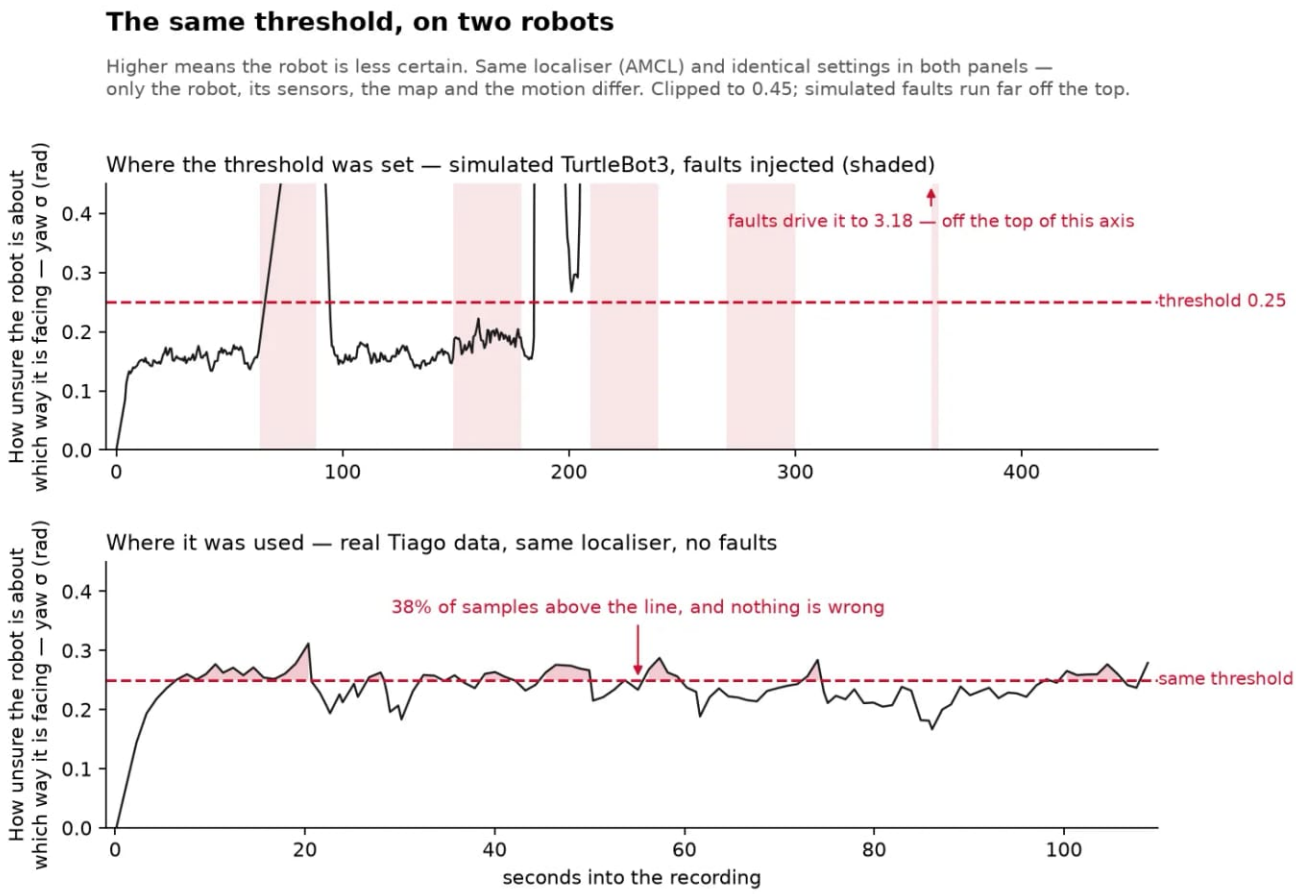
Four independent fault detectors for ROS 2/ROS 1 localization pipelines — sensor dropout, transform jumps, covariance spikes, pose divergence — tested against 108 minutes of real recordings from five robot platforms (Cartographer backpack rigs, a Tiago, a MiR100 AGV, a PR2, a handheld 3D rig), all public third-party benchmark data, not a live customer fleet. Every graded figure recomputes from a committed artifact with one script, `check_numbers.py`; the cross-platform flag rates recompute from the flag count and duration stated in their own row, bag linked. Clone the repo and run it.

WHY HARD

The same frozen thresholds did not transfer. The covariance threshold, set on a simulated TurtleBot3, sits inside a real Tiago's healthy yaw noise and fired five times on a recording with nothing wrong, graded wrong in the case log. A threshold is a property of the machine it was measured on, not of the failure. That is cross-platform transfer, not fleet calibration; calibrating to a live fleet's own robots is what an engagement builds.

WHAT HAPPENED

- 16 of 16 labelled laser gaps found, against labels the tool's author never wrote (the Cartographer dataset's own decade-old ground truth).
- Three predictions were committed before the ERL benchmark run. Two held, including that the laser-gap detector would show no elevation on the engineered failure runs: it fired zero times over 1,013 seconds. The prediction that the transform detector would not rise did not hold.
- Two case-log rows are graded "wrong" and published as such.
- One candidate dataset was rejected over its licence, not for a technical reason.



The same yaw-uncertainty threshold on two robots: set on a simulated TurtleBot3 where injected faults cross it, then applied to a real Tiago with no faults, where the signal rides the line for much of the recording.

ROS 2 · Log Analysis · Fault Detection · Localization · Python

theoalessandro.com/projects/ros2-localization-triage · github.com/thrmnn/ros2-localization-triage

MORE WORK

EPFL VITA Lab · 2023

Autonomous mobile robot navigation on a Segway Loomo



Hidden	Where the people around a robot are and where they are heading, in a crowded indoor space.
Revealed	A full ROS autonomy stack on a Segway Loomo: mapping, planning, obstacle avoidance, and person-following with detection, tracking and trajectory prediction.
Changed	Validated end to end on the physical robot among real pedestrians, not only in simulation, and open-sourced.

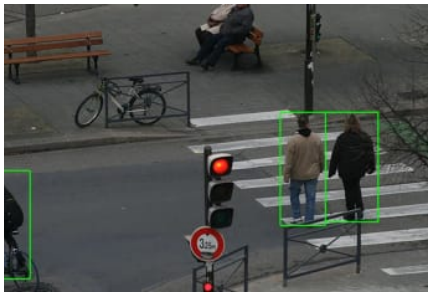
A full autonomy stack — SLAM, planning, obstacle avoidance, person-following — validated end-to-end on a physical robot (Segway Loomo, EPFL VITA Lab research platform).

SLAM · Path Planning · Autonomous Navigation · Multi-Object Tracking · ROS · C++ · OpenCV · Mobile Robotics

theoalessandro.com/projects/autonomous-mobile-robot · github.com/thrmnn/ROS_Autonomous_Driving

Open source · 2026

A real-time vision pipeline



Hidden	Four versions of the same detection and tracking work, built 2022 to 2023, sharing no interface and pinned to stacks that aged out.
Revealed	One modular stack behind narrow interfaces. Version 0.1 ships detection and tracking; re-identification and pose follow.
Changed	A module can be swapped without touching the rest, and every module reports its own latency. Open source, MIT licensed.

Four one-off perception prototypes from 2022 and 2023, consolidated into one modular stack: detection, multi-object tracking, re-identification and pose, each swappable, with a latency budget the whole thing is held to.

Object Detection · Multi-Object Tracking · Re-Identification · Pose Estimation · YOLO · ByteTrack · PyTorch · TensorRT · Edge Deployment

theoalessandro.com/projects/perception-pipeline · github.com/thrmnn/perception-pipeline

IRI (CSIC-UPC, Barcelona) · 2023–24

Event cameras for high-speed drone state estimation

Hidden	Where a drone is during aggressive flight, once frame cameras blur and standard visual-inertial odometry diverges.
Revealed	6-DOF pose from event-camera streams fused with RGB and IMU data, in a PyTorch network trained on high-speed drone flights.
Changed	Pose tracking held through maneuvers above 5 g, the regime where the frame-based pipelines fail.

6-DOF pose estimation through flight maneuvers exceeding 5g acceleration, where conventional frame cameras fail.

State Estimation · Visual-Inertial Odometry · Sensor Fusion · Event Cameras · Deep Learning · PyTorch · ROS · C++

theoalessandro.com/projects/event-cameras-drone · github.com/thrmnn/EVIO

Open source · 2026

A harness for AI agents

Hidden	Agent failures nobody sees: a tool call breaks, nothing traces it, and a user finds out first.
Revealed	A typed tool layer with one structured trace line per call, and a regression suite wired into CI.
Changed	A bad prompt change fails the build instead of reaching the user.

Agentic systems fail the way robots do: a tool call breaks, nothing traces it, and nobody notices until a user does. I built a typed tool layer with one structured trace line per call and a regression suite wired into CI, so a bad prompt change fails the build instead of the user.

AI Agents · Typed Tool Layer · Evals · Tracing · Agentic Systems

theoalessandro.com/projects/agent-harness · github.com/thrmnn/agent-harness